

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 5

Duration : 1 Hour

Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I Mohammad Fuzail / 192565050 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1 – Water Jug Problem

Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and a 3-gallon one. Neither has any measuring marker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into the 4-gallon jug? (Assumptions: a jug can be filled from the pump, water can be poured onto the ground, water can be poured from one jug to another, and no other measuring device is available.)

Problem formulation

- **State:** (x, y) where x = water in the 4-gallon jug ($0 \leq x \leq 4$) and y = water in the 3-gallon jug ($0 \leq y \leq 3$).
- **Initial state:** $(0, 0)$. **Goal state:** $(2, n)$ – exactly 2 gallons in the 4-gallon jug ($n = \text{anything}$).
- **Path cost:** 1 per action (we look for the fewest steps). **Method:** Breadth-First Search (uninformed) over the rules below.

Production rules (operators)

No.	Condition	Action	Result
1	$x < 4$	Fill the 4-gallon jug from the pump	$(4, y)$
2	$y < 3$	Fill the 3-gallon jug from the pump	$(x, 3)$
3	$x > 0$	Empty the 4-gallon jug on the ground	$(0, y)$
4	$y > 0$	Empty the 3-gallon jug on the ground	$(x, 0)$
5	$x + y \geq 4, y > 0$	Pour from 3-gal jug into 4-gal jug until it is full	$(4, y - (4 - x))$
6	$x + y \geq 3, x > 0$	Pour from 4-gal jug into 3-gal jug until it is full	$(x - (3 - y), 3)$
7	$x + y \leq 4, y > 0$	Pour all of the 3-gal jug into the 4-gal jug	$(x + y, 0)$
8	$x + y \leq 3, x > 0$	Pour all of the 4-gal jug into the 3-gal jug	$(0, x + y)$

Solution (sequence of states)

Step	(x, y)	Rule applied / action
0	$(0, 0)$	Initial state – both jugs empty
1	$(0, 3)$	Rule 2 – fill the 3-gallon jug
2	$(3, 0)$	Rule 7 – pour all 3 gallons into the 4-gallon jug
3	$(3, 3)$	Rule 2 – fill the 3-gallon jug again
4	$(4, 2)$	Rule 5 – pour from the 3-gallon jug until the 4-gallon jug is full (1 gallon goes in, 2 remain)
5	$(0, 2)$	Rule 3 – empty the 4-gallon jug
6	$(2, 0)$	Rule 7 – pour the 2 gallons from the 3-gallon jug into the 4-gallon jug → GOAL

Answer: The 4-gallon jug holds exactly 2 gallons after 6 operations: $(0,0) \rightarrow (0,3) \rightarrow (3,0) \rightarrow (3,3) \rightarrow (4,2) \rightarrow (0,2) \rightarrow (2,0)$. An equally short alternative is $(0,0) \rightarrow (4,0) \rightarrow (1,3) \rightarrow (1,0) \rightarrow (0,1) \rightarrow (4,1) \rightarrow (2,3)$.

Question 2 – Mars Rover (PEAS description)

Find out how the Mars Rover analyses and explores samples on Mars by answering: (i) percepts, (ii) characterize the operating environment, (iii) actions, (iv) performance evaluation, (v) most suitable agent architecture and why.

i. Percepts of the agent

- **Cameras / stereo vision:** images of terrain, rocks, sky and sample sites.
- **Spectrometers and chemical sensors:** mineral / chemical composition of rocks and soil, presence of water or organic traces.
- **Proximity / obstacle sensors (lidar, stereo depth):** rocks, craters, cliffs, slopes ahead.
- **Navigation sensors:** wheel odometry, gyroscope, accelerometer, tilt (inclinometer), sun sensor – current position and orientation.
- **Environment sensors:** temperature, wind, dust level, radiation, light.
- **Internal state sensors:** battery / solar-panel power, motor current, wheel slip, arm position, instrument health.
- **Communication percepts:** commands and updates received from Earth (via orbiter).

ii. Operating environment

Property	Mars Rover environment	Reason
Observable	Partially observable	Sensors see only the nearby area; the terrain beyond is unknown, subsurface hidden
Agents	Single agent	Only the rover acts (Earth operators only send high-level goals)
Deterministic?	Stochastic	Wheel slip, sand, dust storms and sensor noise make outcomes uncertain
Episodic?	Sequential	Current moves affect later position, energy and safety
Static?	Dynamic	Weather, dust, lighting and temperature change while the rover acts
Discrete?	Continuous	Positions, speeds, temperatures, time are continuous values
Known?	Largely unknown	Terrain and sample properties are not known in advance

iii. Actions (actuators)

- Move forward / backward, turn, climb, stop; select a path and avoid obstacles.
- Extend the robotic arm, scoop soil, drill / grind rock, collect and store a sample.
- Take photographs, run spectrometer / analysis instruments on the sample.
- Transmit data to Earth and receive commands (antenna).
- Adjust solar panels, recharge, sleep / enter safe mode, heat itself in cold nights.

iv. Performance measure

- Number, variety and scientific value (quality) of samples collected and analysed.
- Area of terrain explored and number of interesting targets reached.
- Amount of useful, correct data successfully transmitted to Earth.
- Safety – rover is not damaged, stuck, tipped over or out of power; mission lifetime.
- Efficiency – low energy and time used per sample / per metre travelled.

v. Suitable agent architecture and why

A **goal-based agent with a utility-based decision component**, built on a **model-based** core and a thin **reflex layer** for emergencies (a hybrid / layered architecture):

- **Model-based:** the environment is only partially observable, so the rover must keep an internal map and state (position, terrain, battery) that is updated from percepts.
- **Goal-based:** its work is goal driven – “reach target rock”, “collect sample”, “analyse and transmit” – which needs planning and search over action sequences.
- **Utility-based:** it must trade off competing aims (science value vs. risk vs. energy vs. time) and choose the best-valued plan, not just any plan that reaches the goal.
- **Reflex layer:** instant actions (stop at a cliff edge, safe mode on low power) cannot wait for planning.
- **Autonomy:** Earth–Mars signal delay (4–24 min) rules out step-by-step remote control.

Question 3 – 8-Queens Problem

Explore the search space using search strategies and formulate the problem components for the 8-Queens problem: place 8 queens on a chessboard such that none of the queens attacks any other. The configuration in the figure is not a solution because the queen in the first column is on the same diagonal as the queen in the last column.

Problem formulation (incremental formulation)

Component	Definition
States	Any arrangement of 0 to 8 queens on the 8×8 board (better: arrangements with one queen per column, none attacked)
Initial state	Empty board – no queens placed
Actions	Add a queen to a square in the leftmost empty column such that it is not attacked by any queen already on the board
Transition model	Returns the board with the new queen added to the chosen square
Goal test	8 queens are on the board and no two queens share a row, column or diagonal
Path cost	Not important – only the final arrangement matters (1 per queen placed if needed)

Size of the search space

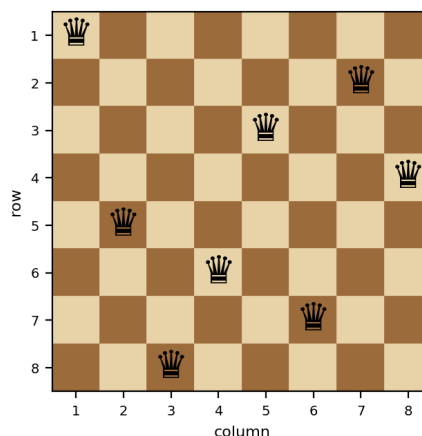
- Naive formulation (place any queen on any empty square): $64 \times 63 \times \dots \times 57 \approx 1.8 \times 10^{14}$ sequences.
- Improved formulation (one queen per column, only non-attacked squares): only **2,057** states – a huge reduction.

Exploring the search space with uninformed strategies

- **Breadth-First Search:** expands level by level (0 queens, 1 queen, ...). Complete, but needs huge memory because it stores all partial boards at each level.
- **Depth-First Search with backtracking:** place a queen column by column; if a column has no safe row, backtrack to the previous column and try its next row. Uses little memory (depth is only 8) and is the natural choice because all solutions lie at depth 8.
- **Why the given figure fails:** goal test is false – column 1 and column 8 queens attack each other diagonally, so the search must backtrack and move one of them.

Valid solution found by DFS / backtracking

Rows of the queens in columns 1 to 8 = (1, 5, 8, 6, 3, 7, 2, 4). All rows are different, and the values of (row – column) and (row + column) are all different, so no two queens share a row or a diagonal. Goal test = TRUE.



Question 4 – OLA Cab Booking

A customer wants to travel from one location to another using the OLA cab booking mobile application. The customer can select any cab type (mini, micro, sedan, shared, prime, etc.) based on comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode.

Type of agent

A **goal-based problem-solving agent** (with a utility function to choose between cab types) is suitable. The environment is fully observable (map, fares and available cabs are shown in the app), deterministic enough, discrete and known, so the agent can *formulate a goal, formulate the problem, search for a solution, and then execute it* (the formulate – search – execute cycle).

Component	For the cab-booking problem
Goal	Customer reaches the destination in the chosen cab (best comfort / cost / time)
Initial state	Customer at pick-up location, no cab booked
Actions	Select cab type (mini, micro, sedan, shared, prime...), confirm booking, driver arrives, travel along a route
Transition model	Result of an action – e.g. Book(sedan) → cab assigned; Drive(route) → location changes
Goal test	Current location = destination
Path cost / utility	Fare + travel time + waiting time, adjusted by customer comfort preference

Pseudocode

```
function CAB-BOOKING-AGENT(percept) returns an action
  persistent: seq    // action sequence, initially empty
                state // current description of the world
                goal  // initially null
                problem // problem formulation

  state <- UPDATE-STATE(state, percept)           // pick-up, drop, cabs, fares
  if seq is empty then
    goal  <- FORMULATE-GOAL(state)                // reach destination D
    problem <- FORMULATE-PROBLEM(state, goal)
    seq    <- SEARCH(problem)                     // see below
    if seq = failure then return NO-CAB-AVAILABLE
  action <- FIRST(seq)
  seq    <- REST(seq)
  return action

function SEARCH(problem) returns a sequence of actions
  pickup <- problem.INITIAL-STATE
  drop   <- problem.GOAL
  cabs   <- GET-AVAILABLE-CABS(pickup)            // mini, micro, sedan, shared, prime ...
  if cabs is empty then return failure
  best <- null ; bestUtility <- -infinity
  for each cab in cabs do
    route <- SHORTEST-PATH(pickup, drop)          // e.g. A* / UCS on road map
    fare  <- ESTIMATE-FARE(cab, route)
    time  <- ESTIMATE-TIME(cab, route)
    utility <- w1*comfort(cab) - w2*fare - w3*time // user preference
    if utility > bestUtility then
      best <- (cab, route) ; bestUtility <- utility
  CUSTOMER-CONFIRM(best)                          // customer may override choice
  return [BOOK(best.cab), WAIT-FOR-PICKUP, DRIVE(best.route), REACH(drop), PAY]
```

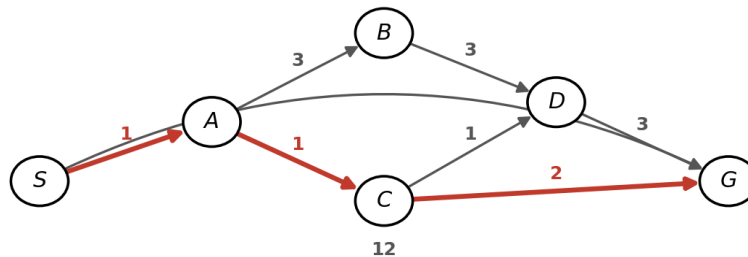
Result: the agent keeps executing the returned sequence (book → pick-up → drive → reach destination → pay); if a percept shows the plan failed (cab cancelled, traffic), it re-formulates and searches again.

Question 5 – Uniform Cost Search (UCS)

A logistics company must find the least-cost path from starting warehouse S to destination warehouse G using Uniform Cost Search on the weighted graph given in the question paper (edge costs = fuel + time).

Problem understanding

Edges (directed, cost): $S \rightarrow A = 1$, $S \rightarrow G = 12$, $A \rightarrow B = 3$, $A \rightarrow C = 1$, $B \rightarrow D = 3$, $C \rightarrow D = 1$, $C \rightarrow G = 2$, $D \rightarrow G = 3$. UCS always expands the frontier node with the lowest path cost $g(n)$ using a priority queue; it is complete and optimal for non-negative edge costs.



Solution process

Step	Node expanded (g)	Frontier after expansion (node : g, parent)	Closed / explored
1	S (0)	A:1 (S), G:12 (S)	S
2	A (1)	C:2 (A), B:4 (A), G:12 (S)	S, A
3	C (2)	D:3 (C), B:4 (A), G:4 (C) – replaces G:12 since $4 < 12$	S, A, C
4	D (3)	B:4 (A), G:4 (C) (G via D = $3+3 = 6 > 4$, ignored)	S, A, C, D
5	B (4)	G:4 (C) (D via B = $7 > 3$, ignored)	S, A, C, D, B
6	G (4)	Goal test passed – stop	S, A, C, D, B, G

Order of expansion: $S \rightarrow A \rightarrow C \rightarrow D \rightarrow B \rightarrow G$.

Interpretation of result

Least-cost path: $S \rightarrow A \rightarrow C \rightarrow G$ with total cost = $1 + 1 + 2 = 4$ units (cost units of fuel + time).

Path	Cost
$S \rightarrow G$ (direct)	12
$S \rightarrow A \rightarrow B \rightarrow D \rightarrow G$	$1 + 3 + 3 + 3 = 10$
$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$	$1 + 1 + 1 + 3 = 6$
$S \rightarrow A \rightarrow C \rightarrow G$ (UCS optimal)	$1 + 1 + 2 = 4$

The direct route $S \rightarrow G$ looks short (one edge) but is the most expensive. Because UCS expands nodes in order of path cost, the first time G is selected for expansion its cost (4) is guaranteed to be the minimum. For the company, routing packages along warehouses S, A, C and G saves 8 cost units (66%) compared with the direct road.